评价器提速方案（供审核）

2026-10-19（审核后修订 v3.1，冻结；3.1 只补缓冲区规则）

一句话。 每一步参考值的非零质量只能落在当前输出 x_k 附近一个固定宽度的窗口里，窗口外在浮点意义下是精确的零；把递推限制在这个窗口上，结果不变（只差求和顺序的舍入），节点数从 43 000 降到 3 265，预计单次评价从 13 s 降到 2–2.5 s。不改数学，不改离散格式，不改 `cfun`。

1. 现状与瓶颈

验收通过的版本 B，$n = 24$：网格范围 $[\min x - 8\Delta,\ \max x + 8\Delta]$，共 $G \approx 43\,262$ 个节点，每步对全部节点做：算 α_i, β_i 和 $\hat{g}_{k,j}$、滑动窗口和 $\hat{P}_{k,i}$、质量更新。CC 的剖析：13.2 s 花在这个 numba 循环上，1.1 s 花在 `cfun`（已只在 $m \neq 0$ 的节点调用）。

但 f_k 在任意时刻都集中在 x_k 附近：参考值要么是上次发送时的 z（离当时的 x 不过几 σ），要么因为离 x_k 太远而马上被发送掉。绝大多数节点上 $m_{k,i}$ 不只是小，而是浮点意义下的精确零。

2. 非零质量的支撑在哪（精确论证）

引理 1（核在实际代码中的精确零——经验前提，不是一般性断言）。 高斯核在 float64 下溢为精确零的位置**依赖运算顺序**：$\mathrm{fl}(e^{-38.6^2/2}/\sqrt{2\pi})/\sigma = 0$，但 $\mathrm{fl}(e^{-38.6^2/2}/(\sigma\sqrt{2\pi})) \approx 1.65 \times 10^{-320} > 0$（审核方复算）。在 39σ 处两种写法都为零。所以不能凭”float64”做一般论证，而要**对已验收评价器实际使用的核表达式和编译配置**（含 numba 的 `fastmath`、并行归约设置，一律保持不变）做检查：

- 单元测试：用代码里的核函数在 $|z - x| = 39\sigma, 40\sigma, 50\sigma$ 处求值，必须精确为 0；在 38.5σ 处记录实际值。
- 通过后，取 $\rho := 39\sigma$；离散核 $\hat{g}_{k,j}$ 在 $|z_j - x_k| \geq \rho$ 处精确为零（分子为零）。

若 39σ 处不为零：报告 39、40、50 三个测试位置中首次为零的位置；三处都不为零则报”未找到”并停止，执行方不自行搜索新阈值。

引理 2（一步之后的支撑）。 版本 A/B 的更新

$$m_{k+1,j} = m_{k,j}\,\hat{P}_{k,j} + \hat{g}_{k,j} \sum_i m_{k,i}(1 - \omega_{ij}), \qquad \hat{P}_{k,j} = \sum_{|i'-j| \leq M} \hat{g}_{k,i'}\,\omega_{ji'}.$$

第二项在 $|z_j - x_k| \geq \rho$ 处为零（引理 1）。第一项：$\hat{P}_{k,j}$ 是窗口 $[z_j - \Delta, z_j + \Delta]$ 内的核质量之和；若 $|z_j - x_k| \geq \Delta + \rho$，窗口内每个核值都为零，$\hat{P}_{k,j} = 0$ 精确，第一项也为零。所以

$$\mathrm{supp}(m_{k+1}) \subseteq \{z :\ |z - x_k| < \Delta + \rho\}.$$

与 m_k 原来在哪无关：离 x_k 超过 $\Delta + \rho$ 的旧质量在第 k 步以概率精确 1 被发送、落回 x_k 附近。

引理 3（相邻两步的位移）。 $|x_{k+1} - x_k| \leq v_{\max} = \Delta t \sqrt{\sum_m (\omega_m |G(j\omega_m)|)^2} = 5.45 \times 10^{-4} = 4.57\sigma$（Cauchy–Schwarz，对族内所有 $\sum a_m^2 = 1$ 成立）。实现用这个族上界取整后的 5σ；给定输入的实际 $\max_k |x_{k+1} - x_k|$ 作为日志核验项记录，不参与窗口定义。

结论（归纳不变量）。 记第 k 步更新前的窗口 $W_k := \{z : |z - x_k| < R\}$，$R := \Delta + 48\sigma$。

- 初始化：$m_2 = \hat{g}_1$ 的支撑在 $|z - x_1| < \rho$ 内，而 $|x_2 - x_1| \le 5\sigma$，所以 $\mathrm{supp}(m_2) \subset W_2$。
- 归纳步：若更新前 $\mathrm{supp}(m_k) \subset W_k$，由引理 2 更新后 $\mathrm{supp}(m_{k+1}) \subset \{|z - x_k| < \Delta + \rho\}$，再由引理 3 $\subset \{|z - x_{k+1}| < \Delta + \rho + 5\sigma\} = \{|z - x_{k+1}| < \Delta + 44\sigma\} \subset W_{k+1}$。

余量 4σ。$n = 24$ 时窗口是闭区间 $[a_k, b_k]$，节点数 $b_k - a_k + 1$ 最多 $2 \cdot 68 \cdot 24 + 1 = 3265$，是全网格的 $1/13.3$。

3. 实现：固定宽度的滑动窗口

网格本身不变（同样的 z_0、h、G，保证节点位置与验收版逐点相同）。每步只操作一个闭索引区间 $[a_k, b_k]$，$a_k = \lceil (x_k - R - z_0)/h \rceil$，$b_k = \lfloor (x_k + R - z_0)/h \rfloor$（Python 切片 `a_k:b_k+1`）。

每步做的事与现在完全相同，只是数组切片从 `[0:G]` 变成 `[a_k:b_k+1]`：

1. 在 $[a_k, b_k]$ 上算 α, β、$g_k(z_j)$；核归一化 $\sum_j g_k(z_j)$ 只在区间内求和——区间外 g_k 精确为零（引理 1，区间半宽 $R > \rho$），所以和值与全网格求和只差求和顺序的舍入。
2. 前缀和、滑动窗口和 $\widehat{P}$：在 $[a_k - M,\ b_k + M]$ 上做（窗口和要用到区间外 M 个节点的核值，它们是零，但索引要存在）。
3. 质量更新：只写回 $[a_k, b_k]$。区间外的 m 不碰——由 §2 的归纳不变量，更新前它们就是零；本步只向 W_k 写入，所以更新后 W_{k+1} 之外可能的非零值只能在 $W_k \setminus W_{k+1}$ 里（见 §4 的断言）。
4. $\mathbb{E}[N]$ 累加、w_k（`cfun` 在 $m \neq 0$ 的节点上，照旧）、质量和、$\min m$、核因子：都在区间内算。
5. 日志加三列：窗口起止索引、区间内非零节点数（初始化那一行单独标出）、本步实际位移 $|x_{k+1} - x_k|/\sigma$。

缓冲区规则。已验收版用两个质量数组轮换：第 k 步从源缓冲区读 m_k，把 m_{k+1} 写进目标缓冲区，下一步对调。窗口版保留两个缓冲区，但目标缓冲区里还存着 m_{k-1}，按不变量其非零值只在 W_{k-1} 内；所以写入前先把目标缓冲区的 $[a_{k-1}, b_{k-1}]$ 清零，再写 $[a_k, b_k]$。这样目标缓冲区在写入后窗口外精确为零，断言才有意义。源缓冲区在本步内不被修改，它本身就是”更新前的 m_k 副本”，§4 的回退直接从它重算，不需要额外复制。初始化时两个缓冲区全零，m_2 写进 W_2。

末步规则。递推步 $k = 2, \ldots, K$，共 4999 步；已验收版在 $k = K$ 时仍执行一次质量更新（得到的 m_{K+1} 不再使用）。窗口版保持同样的更新顺序：$k = K$ 那步的断言检查”更新后的支撑是否在 W_K 内”（没有 x_{K+1}，不定义 W_{K+1}），位移一栏记 `not applicable - no next sample`。不补采样点。

v0 和 A 不改：它们只作对照，验收已完成；提速只做 B。

4. 精确性与安全检查

为什么结果不变。全网格版和窗口版做的是同一组浮点运算，区别只是全网格版多做了许多”零乘零、零加零”。唯一的差异来源是求和顺序：numba 的归约在不同长度数组上可能以不同顺序累加，造成最后一两位的差别。所以回归阈值定为相对 10^{-12}，而不是逐位相等。

运行时断言（每步，依赖归纳不变量）。更新前 W_k 之外全零，本步只写 W_k，所以更新后 W_{k+1} 之外可能残留的非零值只能在 $W_k \setminus W_{k+1}$ 内。相邻窗口端点最多移动 $5\sigma/h = 5n$ 个节点，而 $M = 20n$，所以只需检查 W_{k+1} 两侧紧邻各 M 个节点：

$$\text{断言：}\quad m_{k+1,i} = 0 \ \text{对所有}\ i \in [a_{k+1} - M,\ a_{k+1}) \cup (b_{k+1},\ b_{k+1} + M].$$

检查的是”是否存在非零元素”（`any(m != 0)`），不是带符号的和是否为零（避免正负抵消）。这不是对整个数组的独立检查，它的有效性以不变量为前提，这一点在报告里写明。

若断言失败：**从本步更新前的 m_k 副本**重新做一次全网格更新，替换本步结果；本步已累加的 $\mathbb{E}[N]$ 增量、w_k、质量日志作废并用全网格结果重写（不能在已更新的数组上再更新一次，也不能重复累计）；记日志。全网格重做**只用于诊断**：记录后整次运行停止，不再继续窗口路径，不进入计时；任何一次断言触发即判回归失败。断言触发次数为 0 是验收条件，不是预期。

边界情形。窗口碰到网格两端（$a_k < 0$ 或 $b_k > G - 1$）时截到网格内；验收版的网格范围 $\min x \pm 8\Delta$

远大于 $R = \Delta + 48\sigma = 3.4\Delta$，不会发生，但代码要处理。

5. 回归验收（必须全过，否则不进入后续步骤）

1. **验收评价逐一回归**（v0、A 不改故跳过；B 的 12 次 = 4 输入 × 3 网格，覆盖正式验收实际用过的全部网格）。与已验收的全精度结果比较，容差跑前冻结、按量的性质分三类：
   - 远离零的正量（$\mathbb{E}[N]$ 两种、S、$\log\det J$）：相对差 $\le 10^{-12}$；
   - J 的 81 个元素、w_k 的 5000 个元素：绝对加相对容差 $|\Delta| \le 10^{-12}\,(|\mathrm{ref}| + \|J_{\mathrm{ref}}\|_{\max})$，$w_k$ 用 $|\Delta| \le 10^{-12}(|w_{k,\mathrm{ref}}| + 1)$；
   - 质量守恒：不要求舍入残差相对复现，只要求窗口版自己的每步 $|\sum m - 1| \le 10^{-10}$（原门槛）。
2. **单元测试 (b)、(c) 重跑**，结果不变。
3. **尾部检查重跑**（B，$n = 12$，裕量 16Δ 对 8Δ）：比较的量与第 1 条相同（$\mathbb{E}[N]$ 两种、S、$\log\det J$、J 元素、w_k），容差取第 1 条三类定义中的系数 10^{-12} 改为 10^{-11}。
4. **断言统计**：12 次评价 × 4999 步，断言触发次数必须为 0（验收条件）。
5. **耗时**：同一硬件、同样的稳态测法（去掉 JIT），计时包含每步的断言检查和目标缓冲区的清零；每个评价测 3 次取中位数，报 B/$n = 24$ 四个输入的中位数与最大值。搜索成本按实测折算（每次 2.5 s 时两万次约 13.9 h），不按预测。

6. 预测（在 §2 的核零值测试之前单独提交一次 commit，报告引用其 hash）

编号	预测
S1	12 次回归全部通过，最大相对差 $< 10^{-13}$（只有求和顺序差异）
S2	区间内非零节点数（初始化与后续更新分开报）：初始 m_2 的支撑宽 $< 78\sigma$（$n = 24$ 时 < 1872 个节点）；后续每步 $\le 118\sigma$（≤ 2832 个）。这两个都是上界；下限不预测（下溢和归一化可能进一步减少非零节点）。节点数只作预测对照，不是回归验收条件
S3	B/$n = 24$ 单次耗时中位数 ≤ 2.5 s（numba 循环从 13.2 s 降到约 1 s，`cfun` 的 1.1 s 不变成为主项）
S4	耗时随 n 线性：$n = 12$ 约 1.3 s，$n = 6$ 约 0.7 s
S5	尾部检查差异与验收版同量级（$\le 10^{-11}$）

7. 可选的第二步（本轮不做，列出供决定）

`cfun` 现在是 Python 调用，提速后占一半时间。把它移植进 numba 能再省约 1 s，但 `cfun` 是过审代码，移植等于改动它；本轮不做。先完成窗口版的回归和计时，再判断是否值得。将来若做，需事先定义一致性标准（随机 10^6 组 (α, β) 含尾部、相对差 $\le 10^{-15}$ 是统计抽样一致，不是逐位比对），并把两个实现都保留。

8. 需要审核方判断的点

1. 引理 1 改为”对实际核表达式和编译配置的单元测试”后，是否可接受。
2. §5 三类容差的具体取法。

3. 断言的不变量依赖性是否需要在报告中更显式地声明。